DSA0402-FUNDAMENTALS OF DATA SCIENCE

Technical Presentation

Logistic Regression with a Real-World Example

Guided by:
Dr. Umarani P

Presented by:
G Chandravardhan Reddy
S Aman Hussain
V Shiva

WHAT IS LOGISTIC REGRESSION?

- A supervised learning method used for classification problems
- Models the probability that an input belongs to a particular class
- The output is always a value between 0 and 1
- A threshold (usually 0.5) turns the probability into a class label

1 Input features

For example credit score, income and debt ratio

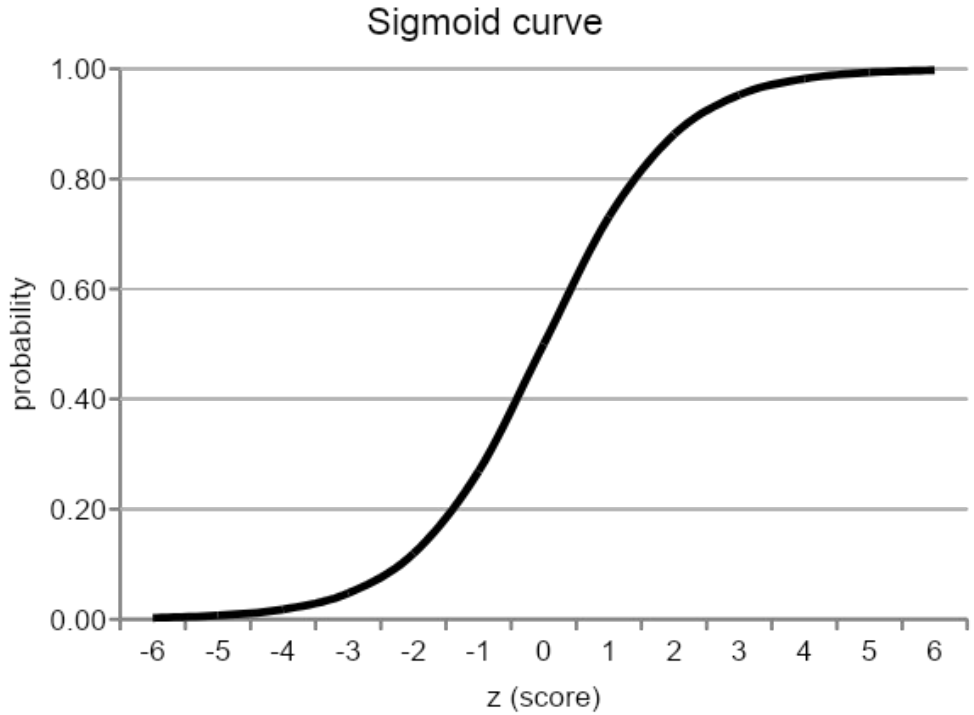
2 Probability

The model outputs a probability between 0 and 1

3 Class label

A threshold turns it into a yes / no decision

THE SIGMOID FUNCTION



$$\sigma(z) = 1 / (1 + e^{-z})$$

- Maps any real number z into the range (0, 1)
- z = 0 gives exactly 0.5
- Large positive z gives a value near 1; large negative z gives near 0
- Smooth S-curve that is easy to differentiate

ODDS, LOG-ODDS & THE MODEL

1 Score

$z = \beta 0 + \beta 1 x1 + \ldots + \beta n xn$

A weighted sum of the input features

2 Probability

$p = 1 / (1 + e^{-z})$

The sigmoid turns z into a probability

3 Log-odds (logit)

$\ln(p / (1 - p)) = z$

Log-odds equal the weighted sum

Odds = $p / (1 - p)$. If $p = 0.8$, the odds are 4 : 1 in favour of the event.

$e^{\beta j}$ is the odds ratio: how the odds multiply for every +1 unit increase in xj.

TRAINING THE MODEL

Log loss (cross-entropy)

$$J = -(1/m)\ \Sigma\ [\ y \cdot \ln(\hat{y}) + (1 - y) \cdot \ln(1 - \hat{y})\ ]$$

- Confident wrong predictions are punished heavily
- The loss is convex, so there is one global minimum
- No closed-form solution, so coefficients are found iteratively

1. Start with zero or random coefficients β
2. Predict probabilities with the sigmoid
3. Measure the error using log loss
4. Update: $\beta = \beta - \alpha \cdot \partial J/\partial \beta$
5. Repeat until the loss stops falling

REAL-WORLD EXAMPLE: LOAN DEFAULT

Goal: predict whether a bank loan applicant will default (1) or repay (0)

Fitted model

z = 4.0 − 0.008 × CreditScore + 0.06 × DTI

P(default) = 1 / (1 + e^−z)

DTI = debt-to-income ratio (%)

Applicant A

Score 650, DTI 40%

z = 1.2

76.9%

High risk: flag for review

Applicant B

Score 780, DTI 20%

z = −1.04

26.1%

Low risk: approve

Reading the coefficients: every +10 credit-score points multiplies the odds of default by e^−0.08 ≈ 0.92 (about 8% lower); every +1 point of DTI multiplies them by e^0.06 ≈ 1.06.

BUILDING THE MODEL: WORKFLOW

1

Prepare Data

Fix missing values, encode and scale features

2

Split Data

80% train, 20% test, keep the class ratio

3

Train Model

Fit the coefficients by minimising log loss

4

Evaluate

Check precision, recall, AUC; tune threshold

Python (scikit-learn)

```
from sklearn.linear_model import LogisticRegression
model = LogisticRegression()
model.fit(X_train, y_train)
prob = model.predict_proba(X_test)[:, 1]
pred = (prob >= 0.5).astype(int)
```

EVALUATING THE MODEL

	Predicted Default	Predicted Repay
Actual Default	**80 (TP)**	**40 (FN)**
Actual Repay	**20 (FP)**	**860 (TN)**

Test set: 1,000 applicants. Lowering the 0.5 threshold catches more defaulters but raises false alarms. ROC-AUC compares performance across all thresholds.

94%

Accuracy

correct overall

80%

Precision

flagged who truly default

66.7%

Recall

defaulters we catch

72.7%

F1 score

precision-recall balance

TYPES, ASSUMPTIONS & USES

Types

- Binary: two classes (spam / not spam)
- Multinomial: 3+ unordered classes
- Ordinal: ordered classes (low / medium / high)

Assumptions

- Binary or categorical outcome
- Independent observations
- Little multicollinearity
- Features act additively on the log-odds
- Enough data per class

Applications

- Medical: disease diagnosis
- Email spam filtering
- Banking: credit scoring
- Telecom: churn prediction
- Marketing: click prediction

PROS, CONS & TAKEAWAYS

Advantages

- Simple, fast and easy to implement
- Coefficients read as odds ratios
- Gives probabilities, not just labels

Limitations

- Decision boundary is a straight line
- Sensitive to outliers and correlated features
- Needs feature engineering for complex patterns

Key takeaway

Thank You!

Logistic regression turns a weighted sum of features into a probability, giving a transparent and reliable method for any yes / no prediction.